COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	10/9/2026	**DATE SUBMITTED:**	10/10/2026
NAME:	Charles Vence Espinas	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11 **Containerization**

1. PROCEDURE OUTPUT

1. Create a new repository for this activity.
2. Install Docker and enable the docker socket.

ubuntu

```
# Install Docker and Python dependencies on Ubuntu
- name: Install Docker and Python SDK on Ubuntu
  ansible.builtin.apt:
    name:
      - docker.io
      - python3-pip
      - python3-docker
    state: present
    update_cache: yes
  when: ansible_os_family == "Debian"
```

- In this task, I used the apt module to install docker.io, python3-pip, and python3-docker on Debian-based hosts. The python3-docker package provides the Python SDK that Ansible's Docker modules need to communicate with the Docker daemon. I used the condition when: ansible_os_family == "Debian" to make sure the task runs only on the Ubuntu servers, which are server1 and server2. I also set update_cache: yes to update the package list before installing the packages.

centOS

```yaml
# Set up Docker repository and install on CentOS
- name: Install Docker prerequisites on CentOS
  ansible.builtin.yum:
    name:
      - yum-utils
      - device-mapper-persistent-data
      - lvm2
      - python3-pip
    state: present
  when: ansible_os_family == "RedHat"

- name: Add Docker YUM repository on CentOS
  ansible.builtin.get_url:
    url: https://download.docker.com/linux/centos/docker-ce.repo
    dest: /etc/yum.repos.d/docker-ce.repo
    mode: '0644'
  when: ansible_os_family == "RedHat"

- name: Install Docker Engine on CentOS
  ansible.builtin.yum:
    name:
      - docker-ce
      - docker-ce-cli
      - containerd.io
    state: present
  when: ansible_os_family == "RedHat"
```

```yaml
- name: Fix Python Docker SDK compatibility on CentOS
  ansible.builtin.pip:
    name:
      - "requests<2.32.0"
      - docker
    state: present
  when: ansible_os_family == "RedHat"

# Enable and start Docker service
- name: Ensure Docker service is running and enabled
  ansible.builtin.systemd:
    name: docker
    enabled: yes
    state: started
```

- In this step, I prepared CentOS for Docker installation in four stages. First, I installed the required packages, including yum-utils, device-mapper-persistent-data, lvm2, and python3-pip. Next, I downloaded the official Docker CE repository file into /etc/yum.repos.d/ and installed docker-ce, docker-ce-cli, and containerd.io. I also used pip to install requests with a version below 2.32.0 and the docker package to fix a known compatibility issue with the Python SDK. Finally, I used the systemd module to start the Docker service and enable it to run automatically when the system boots. This also allows Docker to activate its socket for communication.

3. Add to Docker group to your current user.

```yaml
# Add user to docker group
- name: Ensure docker group exists
  ansible.builtin.group:
    name: docker
    state: present

- name: Add user to docker group
  ansible.builtin.user:
    name: "{{ ansible_user }}"
    groups: docker
    append: yes
```

- In this task, I made sure that the docker group exists and added the user stored in ansible_user to that group. I used append: yes to keep the user's existing group memberships instead of replacing them. This allows the user to run Docker commands without using sudo after logging out and logging back in.

4. Create a Dockerfile to install web and DB server.

```
  GNU nano 7.2                                          Dockerfile
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y nginx mariadb-server && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80 3306

CMD service mariadb start && nginx -g "daemon off;"
```

```yaml
# Copy Dockerfile to target hosts
- name: Create build directory on target hosts
  ansible.builtin.file:
    path: /tmp/docker_build
    state: directory
    mode: '0755'

- name: Copy Dockerfile to target hosts
  ansible.builtin.copy:
    src: Dockerfile
    dest: /tmp/docker_build/Dockerfile
```

- In this task, I created a Dockerfile using ubuntu:22.04 as the base image. I installed nginx, which is a web server, and mariadb-server, which is a database server. I also cleaned the apt cache to reduce the image size. I exposed ports 80 and 3306 for web and database connections. The CMD instruction starts MariaDB first and then runs nginx in the foreground to keep the container running. In Ansible, I created the /tmp/docker_build directory on each host with permission mode 0755 and copied the Dockerfile into it. This directory serves as the build context for creating the Docker image.

5. Install and build the Dockerfile using Ansible.

```yaml
# Build image and start container
- name: Build web-db Docker image
  community.docker.docker_image:
    name: web-db-server
    tag: latest
    build:
      path: /tmp/docker_build
    source: build
    state: present

- name: Run container
  community.docker.docker_container:
    name: my_web_db_container
    image: web-db-server:latest
    state: started
    published_ports:
      - "8080:80"
      - "3307:3306"
```

- In this task, I used the docker_image module to build an image named web-db-server:latest from the /tmp/docker_build directory. Next, I used the docker_container module to create and start a container named my_web_db_container using that image. I mapped host port 8080 to container port 80 for web access and host port 3307 to container port 3306 for database connections. I used port 3307 on the host to avoid conflicts with any database service that might already be running on the host.

6. Add, commit and push it to your repository

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab$ git add HOA11
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab$ git commit -m "Finished HOA11"
[main 538af96] Finished HOA11
 3 files changed, 134 insertions(+)
 create mode 100644 ansible-lab/HOA11/Dockerfile
 create mode 100644 ansible-lab/HOA11/inventory.yaml
 create mode 100644 ansible-lab/HOA11/playbook.yml
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab$ git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.63 KiB | 1.63 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:charlesespinas/CPE212_CharlesVenceEspinas.git
   c0340ca..538af96  main -> main
```

Github Link: https://github.com/charlesespinas/CPE212_CharlesVenceEspinas

- In this task, I used git add HOA11 to stage the files in the HOA11 folder. Then, I committed the changes with the message "Finished HOA11". The commit recorded three files, which were Dockerfile, inventory.yaml, and playbook.yml, with 134 insertions. After that, I used git push to upload the changes to

the main branch of my GitHub repository, CPE212_CharlesVenceEspinas. The commit hash 538af96 confirms the specific commit that contains my changes.

inventory.yaml

```
 GNU nano 7.2                                          inventory.yaml
all:
  vars:
    ansible_user: charles-vence-espinas

  children:
    Ubuntu_servers:
      hosts:
        server1:
          ansible_host: 192.168.56.108
        server2:
          ansible_host: 192.168.56.109

    CentOS_servers:
      hosts:
        centOS:
          ansible_host: 192.168.56.104

    linux_servers:
      children:
        Ubuntu_servers:
        CentOS_servers:
```

- In this task, I defined the shared variable ansible_user as charles-vence-espinas for all hosts. I created two groups in the inventory. The Ubuntu_servers group contains server1 at 192.168.56.108 and server2 at 192.168.56.109, while the CentOS_servers group contains centOS at 192.168.56.104. I also created a parent group called linux_servers that includes both groups. This allows me to target all the machines at once when running the Ansible playbook.

playbook.yml

```
 GNU nano 7.2                                          playbook.yml
---
- name: Setup Docker and Deploy Web DB Container
  hosts: all
  become: true
  tasks:

    # Install Docker and Python dependencies on Ubuntu
    - name: Install Docker and Python SDK on Ubuntu
      ansible.builtin.apt:
        name:
          - docker.io
          - python3-pip
          - python3-docker
        state: present
        update_cache: yes
      when: ansible_os_family == "Debian"

    # Set up Docker repository and install on CentOS
    - name: Install Docker prerequisites on CentOS
      ansible.builtin.yum:
        name:
          - yum-utils
          - device-mapper-persistent-data
          - lvm2
          - python3-pip
        state: present
      when: ansible_os_family == "RedHat"
```

```yaml
- name: Add Docker YUM repository on CentOS
  ansible.builtin.get_url:
    url: https://download.docker.com/linux/centos/docker-ce.repo
    dest: /etc/yum.repos.d/docker-ce.repo
    mode: '0644'
  when: ansible_os_family == "RedHat"

- name: Install Docker Engine on CentOS
  ansible.builtin.yum:
    name:
      - docker-ce
      - docker-ce-cli
      - containerd.io
    state: present
  when: ansible_os_family == "RedHat"

- name: Fix Python Docker SDK compatibility on CentOS
  ansible.builtin.pip:
    name:
      - "requests<2.32.0"
      - docker
    state: present
  when: ansible_os_family == "RedHat"
```

```yaml
# Enable and start Docker service
- name: Ensure Docker service is running and enabled
  ansible.builtin.systemd:
    name: docker
    enabled: yes
    state: started

# Add user to docker group
- name: Ensure docker group exists
  ansible.builtin.group:
    name: docker
    state: present

- name: Add user to docker group
  ansible.builtin.user:
    name: "{{ ansible_user }}"
    groups: docker
    append: yes

# Copy Dockerfile to target hosts
- name: Create build directory on target hosts
  ansible.builtin.file:
    path: /tmp/docker_build
    state: directory
    mode: '0755'
```

```yaml
- name: Copy Dockerfile to target hosts
  ansible.builtin.copy:
    src: Dockerfile
    dest: /tmp/docker_build/Dockerfile

# Build image and start container
- name: Build web-db Docker image
  community.docker.docker_image:
    name: web-db-server
    tag: latest
    build:
      path: /tmp/docker_build
    source: build
    state: present

- name: Run container
  community.docker.docker_container:
    name: my_web_db_container
    image: web-db-server:latest
    state: started
    published_ports:
      - "8080:80"
      - "3307:3306"
```

- This is the complete structure of the playbook.yml file combines all the tasks needed to set up Docker and deploy the web and database container on all hosts. I used become: true to give Ansible root privileges when performing tasks that require them. The playbook includes installing Docker based on the operating system, setting up the Docker service, adding the user to the docker group, copying the Dockerfile, and building and running the container. I also used when conditions to check the operating system family, allowing the same playbook to handle both Ubuntu and CentOS correctly.

Dockerfile

```
  GNU nano 7.2                                                  Dockerfile
FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && \
    apt-get install -y nginx mariadb-server && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*

EXPOSE 80 3306

CMD service mariadb start && nginx -g "daemon off;"
```

- This is the final Dockerfile stored in my repository. I used Ubuntu 22.04 as the base image and set the frontend to non interactive mode so the apt installation would not stop to ask for input. I installed nginx and MariaDB in a single RUN layer to help reduce the image size. I also used the CMD instruction to start both services in one container. This allows me to use a single Docker image for both the web server and database.

Output/Execution

```
charles-vence-espinas@workstation:~/CPE212_CharlesVenceEspinas/ansible-lab/HOA11$ ansible-playbook -i inventory.yaml pl
ybook.yml --ask-become-pass
BECOME password:

PLAY [Setup Docker and Deploy Web DB Container] ********************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [Install Docker and Python SDK on Ubuntu] *********************************************************************
skipping: [centOS]
ok: [server2]
ok: [server1]

TASK [Install Docker prerequisites on CentOS] **********************************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]

TASK [Add Docker YUM repository on CentOS] *************************************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]

TASK [Install Docker Engine on CentOS] *****************************************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]
```

```
TASK [Fix Python Docker SDK compatibility on CentOS] ***************************************************
skipping: [server1]
skipping: [server2]
ok: [centOS]

TASK [Ensure Docker service is running and enabled] ****************************************************
ok: [server2]
ok: [centOS]
ok: [server1]

TASK [Ensure docker group exists] **********************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [Add user to docker group] ************************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [Create build directory on target hosts] **********************************************************
ok: [server1]
ok: [server2]
ok: [centOS]

TASK [Copy Dockerfile to target hosts] *****************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]
```

```
TASK [Build web-db Docker image] ***********************************************************************
ok: [server2]
ok: [server1]
ok: [centOS]

TASK [Run container] ***********************************************************************************
changed: [centOS]
changed: [server2]
changed: [server1]

PLAY RECAP *********************************************************************************************
centOS                     : ok=12   changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
server1                    : ok=9    changed=1    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
server2                    : ok=9    changed=1    unreachable=0    failed=0    skipped=4    rescued=0    ignored=0
```

- I ran the playbook using the command ansible-playbook -i inventory.yaml playbook.yml --ask-become-pass. The Ubuntu only tasks were skipped on centOS, while the CentOS only tasks were skipped on server1 and server2. This shows that the conditions worked correctly for each operating system. The Docker service, group, and build tasks reported ok because the hosts were already configured. The final Run container task reported changed on all three hosts which means the containers were created or updated. The play recap showed failed=0 and unreachable=0, confirming that the deployment completed successfully on all three hosts.

Verification

server1

```
charles-vence-espinas@server1:~$ sudo docker ps
[sudo] password for charles-vence-espinas:
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
                  NAMES
15139ab98dff   web-db-server:latest   "/bin/sh -c 'service…"   22 minutes ago   Up 22 minutes   0.0.0.0:8080->80/tcp, 0
0.0.0:3307->3306/tcp   my_web_db_container
charles-vence-espinas@server1:~$ sudo docker exec -it my_web_db_container ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2900  1896 ?        Ss   11:17   0:00 /bin/sh -c se
root          33  0.0  0.0   2900  1956 ?        S    11:17   0:00 /bin/sh /usr/
mysql        128  0.0  4.1 1272776 80632 ?       Sl   11:17   0:00 /usr/sbin/mar
root         129  0.0  0.1   6208  2672 ?        S    11:17   0:00 logger -t mys
root         164  0.0  0.4  55216  9608 ?        S    11:17   0:00 nginx: master
www-data     178  0.0  0.1  55580  3428 ?        S    11:17   0:00 nginx: worker
www-data     179  0.0  0.1  55580  3364 ?        S    11:17   0:00 nginx: worker
root         218  0.0  0.1   7080  3112 pts/0    Rs+  11:40   0:00 ps aux
```



- On server1, I used sudo docker ps to check if my_web_db_container was running from the web-db-server:latest image. I confirmed that ports 8080 and 3307 on the host were mapped to ports 80 and 3306 inside the container. I also used ps aux inside the container to check the running processes. The output showed the mysql process for MariaDB and the nginx master and worker processes, which confirmed that both services started correctly. Finally, I opened http://192.168.56.108:8080 in the browser and saw the “Welcome to nginx!” page, confirming that the web server was accessible from outside the container.

server2

```
charles-vence-espinas@server2:~$ sudo docker ps
[sudo] password for charles-vence-espinas:
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
                  NAMES
bb0422d607b1   web-db-server:latest   "/bin/sh -c 'service…"   25 minutes ago   Up 25 minutes   0.0.0.0:8080->80/tcp, 0
0.0.0:3307->3306/tcp   my_web_db_container
charles-vence-espinas@server2:~$ sudo docker exec -it my_web_db_container ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2900  1824 ?        Ss   11:17   0:00 /bin/sh -c se
root          33  0.0  0.0   2900  1740 ?        S    11:17   0:00 /bin/sh /usr/
mysql        128  0.0  4.4 1338280 86308 ?       Sl   11:17   0:00 /usr/sbin/mar
root         129  0.0  0.1   6208  2824 ?        S    11:17   0:00 logger -t mys
root         165  0.0  0.6  55216 12140 ?        S    11:17   0:00 nginx: master
www-data     175  0.0  0.1  55580  3564 ?        S    11:17   0:00 nginx: worker
www-data     176  0.0  0.1  55580  3564 ?        S    11:17   0:00 nginx: worker
root         221  0.0  0.1   7080  3004 pts/0    Rs+  11:43   0:00 ps aux
```



- On server2, I used sudo docker ps to verify that my_web_db_container was running from the web-db-server:latest image with the same port mappings. I checked the processes inside the container using ps aux and saw the mysql process along with the nginx master and worker processes. This confirmed that the CMD instruction in the Dockerfile started both services. I also visited http://192.168.56.109:8080 in the browser and saw the “Welcome to nginx!” page, confirming that the web server was accessible through server2.

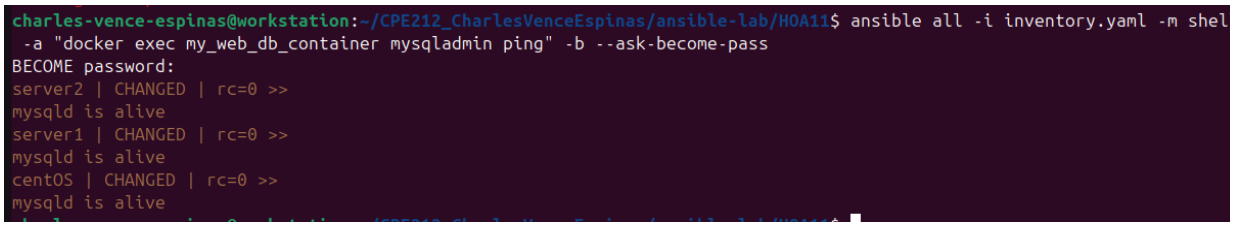
centOS

```
[charles-vence-espinas@centOS ~]$ sudo docker ps
[sudo] password for charles-vence-espinas:
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
495c10fc3bdd   web-db-server:latest   "/bin/sh -c 'service…"   26 minutes ago   Up 26 minutes   0.0.0.0:8080->80/tcp, 0.0.0.0:3307->3306/tcp   my_web_db_cont
iner
[charles-vence-espinas@centOS ~]$ sudo docker exec -it my_web_db_container ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1   2892  1812 ?        Ss   11:17   0:00 /bin/sh -c se
root          33  0.0  0.1   2892  1872 ?        S    11:17   0:00 /bin/sh /usr/
mysql        128  0.0  5.2 1403824 89048 ?       Sl   11:17   0:00 /usr/sbin/mar
root         129  0.0  0.1   6200  2720 ?        S    11:17   0:00 logger -t mys
root         166  0.0  0.7  55208 11996 ?        S    11:17   0:00 nginx: master
www-data     172  0.0  0.1  55572  3296 ?        S    11:17   0:00 nginx: worker
www-data     173  0.0  0.1  55572  3296 ?        S    11:17   0:00 nginx: worker
www-data     174  0.0  0.1  55572  3296 ?        S    11:17   0:00 nginx: worker
www-data     177  0.0  0.1  55572  3296 ?        S    11:17   0:00 nginx: worker
root         217  0.0  0.1   7072  2976 pts/0    Rs+  11:44   0:00 ps aux
```



- On centOS, I used sudo docker ps to confirm that my_web_db_container was running from the web-db-server:latest image with ports 8080 and 3307 mapped to ports 80 and 3306 inside the container. I also used ps aux inside the container and saw the mysql process and several nginx worker processes. This showed that the same Docker image could run on a different operating system family. Finally, I opened http://192.168.56.104:8080 in the browser and saw the “Welcome to nginx!” page, confirming that the CentOS host was correctly exposing the containerized web server.

workstation



- From the workstation, I used an ad-hoc Ansible command with the shell module to run docker exec my_web_db_container mysqladmin ping on all hosts. Each host, server1, server2, and centOS, returned "mysqld is alive" with rc=0. This confirmed that the database service was running correctly inside every container and that I could verify all three hosts from a single control node.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Reflections:
Answer the following:

1. What are the benefits of implementing containerizations?

- The benefits of implementing containerization include consistency across environments, lightweight performance, portability, isolation, and easy maintenance. I learned that containers package an application with its dependencies, allowing it to run consistently on different operating systems, as demonstrated by the web-db-server image running on server1, server2, and centOS. Containers are also lightweight and fast because they share the host's kernel instead of running a complete guest operating system like a virtual machine. I also learned that Docker images can be deployed to multiple servers using Ansible, while container isolation and port mapping help prevent conflicts with other services. Lastly, containers are easy to start, stop, replace, and duplicate, while storing Dockerfiles in Git helps me track changes and manage deployment configurations as code.

CONCLUSION

In this activity, I learned that containerization provides a consistency across different environments by packaging an application with its dependencies which allow it to run the same way on the different operating systems. I also learned that containers are lightweight and fast because they share the host's kernel instead of running a complete guest operating system like a virtual machine. Containerization also makes the deployment easier because I can build a Docker image once and deploy it to multiple servers using Ansible. I learned that containers provide isolation between applications and their dependencies while port mapping allows them to communicate through specific ports. Lastly, I learned that containers are easy to scale, update and maintain while storing Dockerfiles in Git helps me track and manage the configurations as code.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

COMPUTER ENGINEERING

QUEZON CITY

Extent of Use: **I use this tool to help me when I needed an assistance when I am writing a code in ansible**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.